Humour detection in hindi-English code-mixed social media text using transformer based model

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Dhruv Nihate (24U023020)
Sanyam Jain (24U023022)

Under the supervision of

Dr. Ringki Das
Designation of the supervisor
(**Department of Computer Science and Engineering**)

4-November 2026

INDIAN INSTITUTE INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Humor detection in hindi english code mixed social media text using Transformer based Model "** has been satisfactorily completed by **Dhruv nihate and Sanyam Jain (24U023020 & 24U023022)** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-27

Date:

Dr Ringki das,
Assistant professor,
Department of Computer Science,
& Engineering (CSE),
IIIT Bhopal (M.P.)

Head/Department coordinator,
Dr. Rahul Chaurasia,
Department of CSE,
IIIT Bhopal (M.P.)



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled "**Humor detection in hindi english code mixed social media text using Transformer based Model** " presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Ringki Das**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Dhruv Nihate (24U023020) Sign of student1

Sanyam Jain (24U023022) Sign of student2

ACKNOWLEDGEMENT

We, **Dhruv Nihate and Sanyam Jain** would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled " **Humor detection in hindi english code mixed social media text using Transformer based Model** ".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, **Dr. Ashutosh Kumar Singh,** for providing us with the necessary resources and facilities for conducting our project. His/her encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, **Dr. Ringki Das**, for her constant guidance, invaluable suggestions, and expert advice throughout the project. her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, **Dr. Rahul Chaurasia,** for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the **Department of Computer Science and Engineering** for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Dhruv Nihate

Sanyam Jain

24U023020&

24U023022

AREA OF WORK

Our project will mainly focus on BIG DATA and MACHINE LEARNING using a set of tools and technologies that are required for development of large-scale architectures.

While one may argue that the same thing could have been done using big data, but that's where the real catch is. As it is already mentioned that the tools and architecture we'll be using mainly focus on large scale deployments.

It's been a fairly long period since humans started computing and then extracting useful insights from the data, but in past decade the immense growth in data has posed a serious challenge for computing that's where big data came and with time we also acquired the capacity to process real time massive data sets and ever since it's just keeps growing bigger and bigger.

In our project we'll be looking at architecture of real time data pipeline and We'll also look at analysis of big data using spark mlib.

TABLE OF CONTENT

S.no	Title	Page No.
1	Abstract	1
2	Introduction	2
3	Literature review or Survey	3-6
4	Problem defination and objective	7-9
5	Methodology & Work Description	10-17
6	Pruposed algorithm	18-19
7	Pruposed flowchart/block Diagram/DFD	20-23
8	Tools and technology used	24-28
9	Result Analysis	29-33
10	Conclusion & Future Scope	34-35
11	References	36-37
12	Plag Report	38-39

LIST OF FIGURES

Fig	Description	Page no.
1	System block diagram of the proposed Hinglish humor detection system	20
2	Architecture of the XLM-RoBERTa based humor classifier	21
3	Flowchart of the model training process	22
4	Training log (validation loss, accuracy and F1 per epoch),	30
5	Bar chart of weighted F1 for these five rows.	31

LIST OF TABLES

Table No	Description	Page no.
1	Training Configuration	16 -17
2	Hardware and Software Environment	24
3	Environment used for implementation	24-25
4	Tools and libraries and their purpose	25
5	Performace of model	30
6	Classification report on 100 random test samples	30
7	Comparison of models on the test set (1,557 sentences)	31

ABSTRACT

With the growing number of social media platforms, humor and sarcasm became integral parts of online communication. Automatic identification of humorous content is an important task within the field of Natural Language Processing (NLP) and an important step towards creation of conversational intelligent agents. However, there are only several available corpora in English and Hindi that can be used for training models in order to classify whether some text is funny or not. There are many differences between the usage of languages, cultural differences, sentences structure, informal expression etc., that make it a difficult task. Modern developments in the field of NLP and especially appearance of the large language models (LLM) provide opportunities for better understanding of humor and its classification. In the current research, the datasets are analyzed, cleaned and prepared for the training of the model. A pretrained RoBERTa transformer is fine-tuned to detect the humor and test its ability to classify texts into classes: humorous and not humorous.

The result of experiments is F1-score of 0.64 and accuracy of 64% which shows the potential of humor detection while indicating its difficulty.

Additionally, the analysis of the classification errors helps to understand the limitations of the proposed method and areas of further improvement. These findings will be helpful for future research in humor detection by means of NLP and can be used to improve the quality of datasets, architectures of models, their contextual understanding and evaluation metrics.

We've trained and optimized our model from the data publicly available:
https://github.com/Sanyam-Jain27/minor-project.git

INTRODUCTION

As social media and online chat are growing fast texting has become one of the most common ways people express themselves today. People use texting to share ideas, feelings, opinions and jokes with each other. Humor is a part of how people talk to one another because it makes communication lively and interesting.. It is hard to spot humor in text because understanding humor needs many things, such as context, language, cultural references, irony, sarcasm, puns and the meaning that the speaker wants to give. All of these parts make the automatic detection of humor a hard job in the field of Natural Language Processing.

With the progress of Artificial Intelligence and Natural Language Processing it is now possible to build systems that can analyze and classify text. Classification methods let a system group text, into classes based on patterns that it has learned. In humor detection a system can use this method to decide whether a text is humorous or not. These systems can be used for sentiment analysis, social network analysis, conversational AI and many other applications that involve language.

Standard text-processing methods often struggle to understand the context and complicated grammar that go into recognizing humor.. The transformer model has made things better for detecting humor by using attention mechanisms. These mechanisms help the model see how words connect with each other and with parts of the text. Also pre-trained transformers can be fine-tuned for tasks and can use existing language models for classifying humor. Still detecting humor is tough because of the nature of humor differences in language not enough good data sets for the task and many cultural and situational factors.

The issues get even bigger when dealing with English code-mixed text that shows up on social media. This kind of text mixes words, phrases and grammar from both languages along with spelling, slang and meanings that depend on the situation.. When two different languages are, in the same sentence it's really hard for a model to figure out what it means. Also humor and sarcasm often mix a lot when it comes to how language's used.

The objective of this research is to create a transformer-based system that enables the automated identification of humor in text using binary classification techniques. The system will examine texts and categorize them as humorous/non-humorous. The technology employed is an attempt to use the capabilities of transformers for recognizing the language features relevant to identifying humor. Additionally, this study points out difficulties related to informal and code-mixed language being processed and investigates the possible advantages of transformer technology in the field of automated humor detection.

LITERATURE REVIEW

1. Traditional Approaches to Humor Detection:

In the past, most humor detection research depended on linguistic characteristics, handcrafted rules, and traditional machine learning methods. Mihalcea and Strapparava [6] studied automatic humor detection through linguistic conventions and computational methods. Commonly employed traditional text classification techniques include Bag of Words (BoW), term frequency–inverse document frequency (TF-IDF), and n-grams as textual data representations. Salton and Buckley [28] explained the importance of term-weighting techniques in text processing and retrieval systems.

Although the traditional machine learning methods are good at detecting patterns, their efficiency is proportional to the extracted features. Since humor includes several complexities including ambiguity, contextual meaning, irony, and surprising interpretation, surface textual data can prove inadequate for capturing it. This has stimulated research on the use of deep learning and contextual language modeling methods for humor detection.

2. Humor Detection Using Deep Learning:

The use of deep learning techniques allows the models to learn textual representations from data directly. Kim [24] proved that convolutional neural networks can be successfully used for sentence classification as all neural techniques for text processing are based on that idea. Chen and Lee [8] illustrated the application of convolutional networks for humor detection, while Yang et al. [7] conducted the research on humor detection and humor anchor extraction.

Weller and Seppi [9] discussed the use of transformer-based techniques on humor detection illustrating how context-dependent language models can be applied for that goal. Annamoradnejad and Zoghi [23] undertook the same studies in computational humor using BERT-based sentence representations and synaptic neural networks.

Although deep learning models provide the opportunity of learning the most sophisticated patterns and relationships from the textual data, still their performance will depend on the factors such as the quality of the dataset, the characteristics of the domain, language differences, and the distribution of humorous and non-humorous examples.

3. Hindi-English Code-Mixed Text:

Code mixing is a phenomenon that occurs regularly in multi-lingual communication, particularly on social media platforms where people often use multiple languages in a single sentence. One prominent instance of code-mixing is the case of Hindi and English text. Code-mixing in Hindi (Hindi-English) becomes a challenge for natural language processing because Hindi words can be written in Roman script along with English words.

Khandelwal et al. [10] studied the problem of humor detection in code-mixed content on social media. They created a corpus and baseline system, which can serve as a starting point for studying the issue of humor detection in using Hinglish. Sitaram et al. [11] provided an overview of the processing of code-switched speech and language.

Khanuja et al. [12] presented the GLUECoS benchmark that can be used for testing the code-switching methods in natural language processing. Patwa et al. [26] studied the issue of sentiment analysis of code-mixed tweets.

The detection of Hinglish humor is not an easy task due to the use of varied spellings, translation from one language script to another, use of acronyms, casual phrases, use of different grammatical rules, and the meanings depending on context. As a result, this indicates that testing these systems using code-mixed data is required and that there is no guarantee that regular English language data will give the same results.

Example:

Hindi : अंजेलिना एक नए उत्साह के साथ इस फाइनल राउंड में उत्तरी हैं
English : Angelina enters this final round with a new spirit
Code-mixed sentence : angelina एक नए उत्साह के साथ इस final round में उत्तरी हैं

3.4 Transformer-Based Language Models:

The transformer architecture created by Vaswani et al. [3] utilizes mechanisms of attention to create relationships amongst tokens in the sequence. In contrast to other methods that are based mainly on the use of solely regulated features, transformer models are also able to generate representations that take into consideration relationships among words and text that comes before and after them.

BERT is a model based on transformers that was created by Devlin et al. [2], and it is able to learn linguistic representations through the use of concepts such as language modeling. The model is able to create contextual representations that will help to complete tasks through

fine-tuning. RoBERTa created by Liu et al. [5] is an improved version of BERT based on changing the methods of training.

XLM-R was built on the basis of the technology that was implemented in cross-lingual representation learning, and is generated on texts in one hundred languages. Training in multiple languages allows it to be used in connection to different languages. Nonetheless, training does not guarantee competent performance in humor detection using Hindi-English language.

Kudo and Richardson introduced SentencePiece, which is a subword tokenizer and detokenizer that can be applied across all languages. Subword tokenization is useful in multilingual and code-mixed contexts due to its capability to translate fragments that might not be complete words in a dictionary.

Pires et al. studied the multilingual aspects of multilingual BERT, which adds context to comprehending cross-lingual transfer in transformer-based models. Khanuja et al. introduced MuRIL that can provide representation to different Indian languages.

3.5 Humor Detection Datasets and Evaluation:

The development of humor detection systems depends on datasets and reliable evaluation methods.. Strapparava [6] looked at humor recognition while Yang et al. [7] Examined humor. Humor anchor extraction. Khandelwal et al. [10] Focused on EnglishHindi social media content making their work especially relevant to the present study.

Other research has looked at humor in specialized settings. Hossain et al. [20] Presented a dataset and analysis of text editing for humorous headlines. Hossain et al. [21] Examined the assessment of humor in news headlines through SemEval2020 Task 7. Meaney et al. [22] Introduced the Hackathon task, which addressed humor and offense detection and rating.

These studies show that humor detection can have goals, such as binary classification, humor rating finding humorous parts and separating humor from offensive content. This study focuses on classification, where each text is labelled as humorous or nonhumorous.

Model performance should be measured with suitable classification metrics. Sokolova and Lapalme [27] reviewed performance measures, for classification tasks. Therefore accuracy, precision, recall and F1score give views of how well a model works. Using metrics is helpful because accuracy alone may not fully explain a model's ability to spot examples from each class.

3.6 Implementation Frameworks and Training Techniques

Curent transformer-based NLP systems require specialised software libraries for the implementation and training of their models. Paszke et al. [14] mentioned the introduction of PyTorch, a deep learning framework for the implementation of neural network models. Wolf et al. [15] introduced the Transformers library, which gives access to pretrained transformer architectures and the required tools for fine-tuning.

The Datasets library developed by Lhoest et al. [16] aids in loading and processing datasets in an NLP workflow. The Scikit-learn library introduced by Pedregosa et al. [17] provides algorithms for machine learning, data pre-processing, and classification evaluation.

Training efficiency is an essential concern in transformer model fine-tuning. Micikevicius et al. [18] discussed mixed-precision training that maximises computations using suitable hardware to save memory. Loshchilov and Hutter [19] proposed decoupled weight decay regularisation normally used together with the AdamW optimizer that is widely applied in the training of transformer models.

The use of all the mentioned software libraries and methods contributes to the successful application of transformer-based classification systems. Their appropriateness depends on the hardware capabilities, the size of the dataset, the architecture of the model, and the training parameters.

3.7 Research Gap and Relevance to the Present Study

The reviewed literature demonstrates the progression of humor detection from traditional feature-based approaches to deep learning and transformer-based methods. Previous research has examined general humor recognition, neural text classification, multilingual language representations, and code-mixed NLP. However, Hindi-English code-mixed humor detection presents additional challenges because of informal writing conventions, Romanized Hindi, language switching, and the contextual nature of humor.

Although multilingual transformer models such as XLM-RoBERTa provide contextual representations across multiple languages, their effectiveness on a specific Hinglish humor detection dataset must be established through empirical evaluation. General results from multilingual benchmarks or English humor datasets cannot independently establish performance on Hindi-English code-mixed text.

PROBLEM DEFINITION AND OBJECTIVES

4.1 Problem Definition:

The rise of various types of social media has led to the accelerated growth of informal, multilingual, context-dependant communication. People often share their messages, which can be full of humour, irony, sarcasm, and other ways of expression. Identifying humor in such communications has given rise to some interesting problems in Natural Language Processing (NLP).

Detecting humor isn't an easy task since the funny stuff is not always written and doesn't respect the syntax rules. For something to be humorous it can be based on the usage of language, unexpected situations, ambiguous meaning, exaggeration of ideas and connections among different parts of a sentence. Therefore, computer software has to be able to read well enough to distinguish between funny sentences and ordinary ones.

The issue gets tricky when the sentences are in Hindi-English code-mixed linguistic form known as Hinglish. In their everyday communication, users usually mix Hindi and English terms in one sentence. The text may be romanized Hindi, casual spelling, abbreviations, and inconsistent formations, with switching between languages. These features create problems for the common language processing methods and models that are applied with traditional monolingual texts.

The main issue being tackled in this project is how to create and assess a text classification technology based on transformers that can determine if a piece of text is funny or not if it is a Hindi-English code-mixed text. The model utilized in the suggested methodology is the one already trained on multilingual text that can be modified on relevant instances to achieve the classification as necessary.After receiving the text to classify, the system needs to convert the text into tokens so that it is ready for processing through the relevant transformer model already modified.In terms of system evaluation, the standard methods of classification evaluation such as accuracy, precision, and recall are employed.

The aim of this project is to work on text-based humor detection, as the attempt to model all features of humor perception, cultural interpretation or contextual understanding are excluded from consideration.

4.2 Challenges:

However, several challenges remain relevant to the present work.

1. Difficulty of code-mixed text: The code-mixed exchange in Hindi and English is characterized by spelling differences, transliteration, informal grammar, and switching of languages. These properties hinder the representation and classification of real-life text.

2. Contextual meaning of humor: Humor is a function of the words interacting, the message intended by a statement, and context. Simple lexical features might not be adequate to capture such relations.

3. Limits of isolated sentence analysis: A text-based classifier has no access to the speaker's intonation, body language, and previous conversations unless they are given explicitly. It means that it cannot understand humor, which requires information other than that provided in the sentence.

4. Quality of a dataset: The effectiveness of models depends on the quality, consistency, and representativeness of the labelled data. Ambiguous instances and inconsistent labels greatly complicate supervised learning processes.

5. Need for task-oriented evaluation: Improvements seen in multilingual benchmarks or sarcasm datasets cannot be considered to be improvements in Hindi-English humor detection. The choice of model has to be evaluated on its performance on the task of interest, using suitable metrics.

6. Evaluation beyond accuracy: Overall accuracy may not be an adequate measure of effectiveness when the classes are imbalanced or the error differs between humorous situations and non-humorous situations. Hence, precision, recall and F1 score need to be added to the evaluation process.

To address these issues, the present project focuses on testing the utility of XLM-RoBERTa for the task of binary humor classification in Hindi-English code-mixed text. It uses a supervised fine-tuning approach to train the model before evaluating it on relevant classification benchmarks. The project, however, does not claim to address all the cultural and contextual issues related to humor. Instead, it attempts to conduct an experimental study of the utility of a multilingual transformer-based approach to text classification in the specified context.

4.3 Objective:

The primary aim of the project is to create a transformer-based model that can be used to identify humor automatically in social media text written in Hindi-English code-mixed language. This will include using contextual language representations to separate humorous and non-humorous text.

The project has the following specific aims:

- Automating the detection of humor: Create a text classification model that can detect humor in the Hindi-English code-mixed text.

- Data processing: Clean the data and prepare it for usage in the training and evaluation of the model.

- Using a multilingual transformer model that has already been trained: Use the XLM-RoBERTa network to extract contextual features for Hindi-English code-mixed text.

- Creating an effective text tokenization system: Transform sentences into tokens that can be processed in the XLM-RoBERTa model.

-For identifying improvements for future: Look for possible improvements like hyperparameter tuning, better quality of data, and comparing the results of other transformer models.

PROPOSED METHODOLOGY AND WORK DESCRIPTION

1. Introduction:

This research aims to create a transformer-based solution that will automatically identify humor in social media posts using Hindi-English code-mixed language. To accomplish this, the new system employs XLM-RoBERTa, which is a complex multilingual model that can distinguish between humorous and serious posts.

The approach used here is based on supervised learning where a model is trained on previously labeled text segments. In the beginning, the input text goes through a preprocessing stage, where it is taken through representation conversion to obtain token-based representations which are then used during the processing phase in a transformer-based model.

The overall approach consists of processes such as dataset preparation, text preprocessing, dataset splitting, tokenization, model preparation, model training, and evaluation stages.

Each process stage contributes towards building the classification model. The dataset preparation process leads to data uniformity, while tokenization makes it machine-readable using tokens, and fine-tuning ensures that the model can learn relevant patterns.

2. Overview of the Proposed Methodology:

The proposed system follows a sequential workflow consisting of the following major stages.

1. Dataset loading.
2. Data inspection and preprocessing.
3. Preparation of training, validation, and testing subsets.
4. Tokenization using the XLM-RoBERTa tokenizer.
5. Configuration of the pretrained classification model.
6. Fine-tuning using labeled training data.
7. Evaluation using classification metrics.
8. Prediction of humor labels for input text.

The process initiates from a predefined dataset consisting of text examples and class labels. Once the data is pre-processed, it is separated into batches for model training and assessment.The input text undergoes the process of tokenization through the tokenizer used in XLM-RoBERTa. This results in obtaining a set of tokens which are input to the trained transformer model.

During the fine-tuning process of model training, the model gets trained through the training data. The validation samples evaluate the model at the same time while the held-out test data provides the model's classification results.

The overall process can be represented as:

Input Dataset → Data Preprocessing → Dataset Splitting → Tokenization → XLM-RoBERTa Model → Fine-Tuning → Model Evaluation → Humor Prediction

5.3 Dataset Description:

The dataset is a fundamental component of the proposed system because it provides the examples from which the model learns to distinguish humorous text from non-humorous text.

The project uses a CSV file named `hindi_humour.csv`. The dataset contains text samples and their corresponding classification labels.

The two principal columns used by the classification pipeline are:

column	description
text	Contain the hindi-english code mix text
label	Contain the target classification

The text column provides the input to the model, while the label column supplies the expected output during supervised learning

5.3.1 Dataset Inspection:

Dataset inspection involves examining the structure and quality of the available data before applying preprocessing operations.

The main checks include:

- Verifying the presence of the required text and label columns.
- Examining missing values.
- Checking the data types of the columns.
- Inspecting the distribution of the target labels.
- Identifying duplicated text entries.
- Checking for inconsistencies in the classification labels.

5.3.2 Importance of Label Quality :

In supervised classification, it is highly influenced by the quality of the labels used in training datasets.

For example, if a statement is labeled as funny when in real sense the annotator's criteria thinks it is not funny, then the model receives conflicting message during training period.

Such inconsistencies may hinder the ability of the model to learn solid decision boundaries. Therefore, the process of checking labeling consistency is crucial when preparing the dataset for humor detection.Furthermore, it is necessary to examine thoroughly any duplicates texts with contradicting labels, as they may signal the inconsistency of the annotation process or ambiguities in the definition of humor.

5.4 Data Preprocessing:

Preparing unprocessed datasets for application in machine learning pipelines is referred to as data preprocessing. The aim of preprocessing is to increase the consistency of data while retaining crucial linguistic features necessary for humour detection.

1. Handling missing values
2. Text cleaning
3. Duplicate deletion

5.5 Dataset Splitting:

After preprocessing, the dataset is divided into subsets for training, validation, and testing.

These subsets serve different purposes in the model development process.

Training Set: The training set contains the examples used to fine-tune the pretrained XLM-RoBERTa model.

Validation Set: The validation set is used to assess model performance during development. It helps monitor how well the model performs on examples that were not used directly to update its parameters.

Test Set: The test set contains held-out examples used to assess the final model.

The use of stratified splitting in classification tasks is worthwhile since it preserves the approximate distribution of target classes in the subsets being formed. The report must include accurate information about the splitting ratios, random seed, and the stratification parameters used in the experiment.

6. Tokenization uisng XML roBERTa:

Tokenization turns the text into a string of tokens which might be used in the transformer model.Natural language sentences can't be treated by computers as strings of text in the neural network, so the tokenizer performs the conversion of a text into numbers which point to the vocabulary associated with the model.

The tokenizer the project is based on is the one for xlm-roberta-base.

The difference between a regular tokenization using only white spaces and tokenization in transformers is that the latter may decompose the word into the subword units. This way all words and phrases that do not fall into one of the entries of the vocabulary can be represented.

Subword tokenization works very well in cases of informal speech, because they might be used in non-standard cases such as uncommon spelling, transliteration, and the mixture of Hindi and English vocabulary.

1. Token IDs:

The tokenizer converts an input sentence into a sequence of token IDs.

For example, consider the illustrative input:

"Bhai, exam bahut tough tha!"

The tokenizer converts this sentence into a sequence of tokens and maps those tokens to numerical identifiers in the model's vocabulary.

The token sequence produced by or constructed by each tokenizer varies with the tokenizer and its vocabulary. Thus, the use of an example here exemplifies the process and doesn't show the real tokenization from the model itself.

The output token IDs give the necessary numerical data to be fed into the transformer.

2. Attention Masks:

An attention mask signifies what positions in the tokenized input are actual input tokens and what positions are used for padding.

This is significant when multiple sentences are processed in the same batch, and they have different lengths.The attention mask assists the model in differentiating the valid input positions from the padding positions when computing attention.

3. Padding and Truncation:

The length of text samples can vary, but it is required that batches of input sequences are evaluated in regions of uniform size.They insert specialized padding tokens into shorter sequences while truncation consists of the removal of characters that exceed the chosen maximum length.

Maximum sequence **length of 128** tokens is employed in the tokenization pipeline of the project.

This implies that there is an invariable upper limit of the length of the processed inputs. However, truncation may result in the loss of content in lengthy sentences, thus the length could be seen as a technical solution rather than a strict assurance that nothing goes missing.

4. Output of Tokenization:

The tokenization stage prepares the input features required by the transformer model.

Depending on the implementation, the resulting features include:

- `input_ids`: Numerical identifiers representing the tokens.
- `attention_mask`: Values identifying valid input tokens and padding positions.
- `labels`: The target class associated with each text sample.

6. XLM-RoBERTa Model Architecture:

XLM-RoBERTa is a multilingual pretrained transformer model introduced on the basis of learning contextual representations of texts across many languages.
The xlm-roberta-base checkpoint is used in this project and adjusted to the task of binary humor classification.

The structure of the model comprises three major elements which are the tokenizer, the transformer encoder, and the classification head.

1. Input Representation:

Initially, the sentence has been transformed into token IDs using the XLM-RoBERTa tokenizer.Internal representation is done with the token IDs so that the model can effectively process the internal representation of the token. Positional information is utilized as a measure of tracking where the tokens are located in the input.

After gaining such representation, it forms the input to the transformer encoder.

2. Transformer Encoder:

The transformer encoder is made up of many layers that leverage self-attention and feed-forward processes to create representations that take context into account.Self-attention enables the representation of a particular token to take into account information from other related tokens in the sequence of the input.

For example, the contribution of a word to a sentence's meaning can depend on another word present somewhere later in the same sentence. Contextual representation enables the first word's representation to rely on information provided by the neighboring tokens
This is advantageous for classification tasks because a phrase's meaning can rely on relationships between words rather than the meanings of each individual word per se.

The model XLM-RoBERTa takes representations obtained during multilingual pre-training as its basis when performing classification tasks.

3. Classification Head:

The pretrained transformer is configured with a sequence classification head.

The classification head converts the representation produced for the input sequence into scores corresponding to the target classes.

In this project, the model is configured with two output classes:

- Class 0: Non-humorous.

- Class 1: Humorous.

The output scores are used to calculate the classification loss during training and generate predictions during inference.

The model learns task-specific patterns by updating its trainable parameters during fine-tuning.

5.7.4 Why XLM-RoBERTa Is Used:

The reason for choosing XLM-RoBERTa is because we are dealing with Hindi-English code-mixed texts and not just texts in the English language.

Thanks to its multilingual pre-training, the model provides the basis for the representation of texts containing vocabulary from different languages.In addition, the model can be fine-tuned based on labeled data from target datasets, allowing the previously acquired knowledge to be adapted for humor identification purposes.

However, multilingual pre-training does not mean that the model will always understand humor, as the performance depends on the quality of the dataset as well as on the type of assessment of linguistic patterns used during the training.

For this reason, the choice of XLM-RoBERTa has to be seen as a way of modeling that is investigated by the project.

8. Model Training and Fine-Tuning :

Training model refers to modifying the existing XLM-RoBERTa model so that it may perform a task of humor classification.

Rather than starting from scratch by training a massive language model, we start from an existing pretrained model and make use of it to adapt the model by using labeled training data.

This way, we take advantage of the fact that model has acquired knowledge during multilingual pretraining and is able to learn task-specific decision boundaries.

Table1-Training Configuration:

Parameter	Configured value
Pretrained model	Xlm-roberta-base
framework	pytorch

Training interface	Hugging face
Learning rate	(2 x 10^{-5})
Training batch size	16
Validation batch size	16
Maximum sequence length	128
Evaluation strategy	Epoch based
Saving strategy	Epoch based
Best model selection	Enabled in configuration
Classification classes	2

PROPOSED ALGORITHMS

ALGORITHM:

Algorithm 1: Data preprocessing and dataset splitting -

Input : Raw dataset D with columns (text, label)

Output: Training set, Validation set, Test set

1. Load D from hindi_humour.csv
2. Remove every row in which text or label is missing
3. Remove leading and trailing spaces from every text
4. Remove rows whose text is empty
5. Split D into Train (80%) and Temp (20%),
   stratified on label, random seed = 42
6. Split Temp equally into Validation (10%) and Test (10%),
   stratified on label, random seed = 42
7. Return Train, Validation, Test

Algorithm 2: Fine-tuning XLM-RoBERTa for humor classification -

Input : Train set, Validation set, pre-trained XLM-RoBERTa-base (M),
tokenizer (T), max length L = 128, batch size B = 16,
epochs E = 4, learning rate = 2e-5, weight decay = 0.01

Output: Best fine-tuned model M*

1. Tokenize every sentence in Train and Validation using T,
   truncating to L tokens
2. Add a new classification head with 2 outputs on top of M
3. best_F1 = 0
4. For epoch = 1 to E do
   1. Shuffle Train and divide it into batches of size B
   2. For each batch do
      a. Pad the sentences to the longest one in the batch
      b. Pass the batch through M to get scores (logits)
      c. Convert the scores to probabilities using softmax
      d. Calculate the cross-entropy loss against true labels
      e. Back-propagate the loss
      f. Update the weights using the AdamW optimizer
   3. Evaluate M on Validation: compute accuracy and weighted F1
   4. Save a checkpoint
   5. If weighted F1 > best_F1 then
      best_F1 = weighted F1, M* = current model
5. Return M*

Algorithm 3: Model evaluation and humor prediction -

Input : Best model M*, Test set, new sentence s
Output: Performance metrics, predicted label for s

Part A: Evaluation
1. Tokenize every sentence in the Test set (max length 128)
2. For each sentence, obtain predicted label = argmax of M* output
3. Compare predicted labels with true labels
4. Compute accuracy, precision, recall and weighted F1-score
5. Report the metrics and the classification report

Part B: Prediction on new text
6. Tokenize s with truncation to 128 tokens
7. Pass s through M* and apply softmax to get probabilities
8. Label = class with the highest probability
9. If label = 1 return "Humorous", else return "Non-Humorous"

Mathematical Formulas Used:
1. Self-attention (inside the encoder layers): $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$
2. Softmax: $P(\text{class } i) = e^{(z_i)} / (e^{(z_0)} + e^{(z_1)})$, where z are the two scores from the classifier
3. Cross-entropy loss: $L = -(1/N) \Sigma \log P(\text{true class of each sentence})$
4. Accuracy = (TP + TN) / (TP + TN + FP + FN)
5. Precision = TP / (TP + FP)
6. Recall = TP / (TP + FN)
7. F1-score = 2 × Precision × Recall / (Precision + Recall)
8. Weighted F1 = the F1 of each class multiplied by that class's share of the test set, then added together

PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

System block diagram of the proposed Hinglish humor detection system :

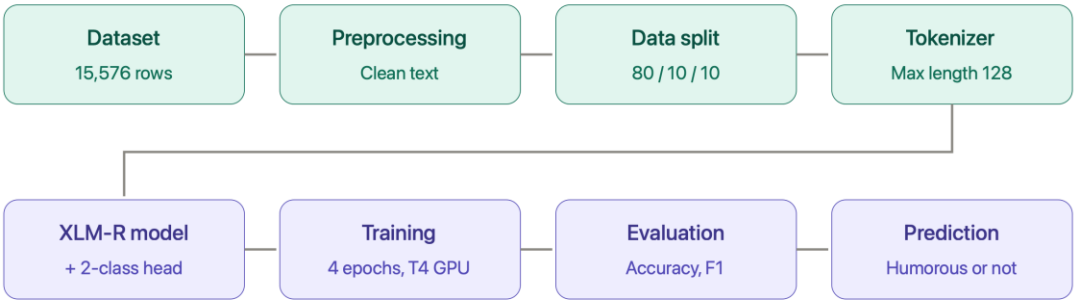


Fig1: flow chart of pruposed model

1. Dataset: The first step involves gathering data. This data is the hindi_humour.csv file. It contains 15,576 labeled sentences that are either classified as humorous or non-humorous.

2. Preprocessing: Next, the data must be prepared for the process. For this, we first remove rows with either missing or empty text. Next, we strip any excess whitespace from each sentence. The final count for the processed sentences is 11,675.

3. Data split: We split our data into training, validation, and testing sets, in the ratio of 80%, 10%, 10% respectively, while also maintaining the same ratio of humorously labeled data versus non-humorous data in all of the sets. As a result, the training set, validation set, and testing set have 9,340, 1,167 and 1,168 sentences.

4. Tokenizer: To begin the machine learning phase, we tokenize each sentence into subwords using XLM-RoBERTa, setting the max length of each sentence to 128 tokens.

5. XLM-R model: We employ a pre-trained XLM-RoBERTa model with an additional two-class classification layer, with which we perform binary classification of either humorous or non-humorous text.

6. Training: The system learns the differences in humorous versus non-humorous text by going through 4 iterations on a Tesla T4 GPU.

7. Evaluation: The performance of the trained model on test data is measured using both accuracy and F1 score.

8. **Prediction:** Any new sentence that is fed into the system is classified as either humorous or not.
The first four blocks prepare the data and the last four build, train and use the model.

The first four blocks prepare the data and the last four build, train and use the model.

Architecture of the XLM-RoBERTa based humor classifier:

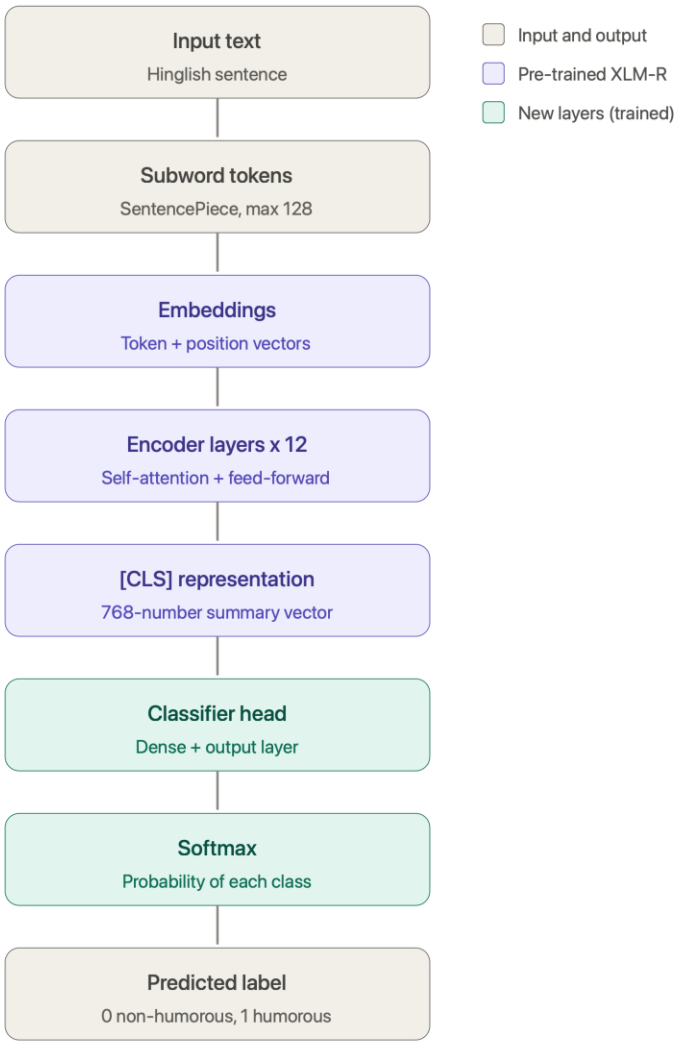


Fig2: architecture of XML-roBERT

The model splits sentences into smaller chunks to help deal with multiple languages (Hindi and Romanized Hindi/English) by creating a single vocabulary. After that, each token is given an embedding that encompasses the word and its position within a sentence.

From here, each embedding passes through 12 transformer encoder layers, in which the model understands the word context with self-attention. With self-attention, each word is able to view other words within the sentence, in addition to being processed through a feed-forward neural network to understand context, like how a setup relates to the punchline.

Once through the final transformer encoder layer, the [CLS] token vector is taken from the vector representing the whole sentence (768 units). The vector then moves on to the classifier head, a combination of two layers that output a couple scores. With the help of the softmax function, these scores are converted into probabilities. The final predicted label is the one that achieves the highest probability.

The embedding and encoder layers of this model start out pre-trained. It is the classifier head, that when trained, fine tunes the whole model, which then gets applied to the specific task dataset.

Flowchart of the model training process:

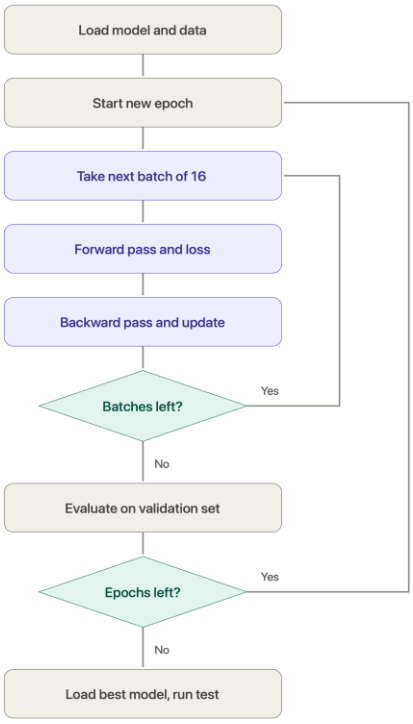


Fig3:flow chart of training process

The process begins by loading the -trained model and the training data. Then a new epoch. The model processes one batch of 16 sentences at a time. For each batch the model makes a pass and the loss is calculated.

The loss shows how wrong the predictions are. Next a backward pass determines how each weight contributed to the error and the weights are updated. The first decision, "Batches left?" checks if there are batches, in this epoch. If yes the next batch is. Processed. With 9,340 training sentences one epoch has 779 steps.

When all batches are done the model is evaluated on the validation set. The accuracy and F1-score are recorded. The second decision, "Epochs left?" checks if all four epochs are complete. If not a new epoch begins. The model saves a checkpoint whenever it achieves the validation F1-score.

After the epoch the best saved model is loaded. This model is then tested on the test set to get the results.

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

This section shows the tools, libraries and environment that were used to build the system with the steps taken to implement it. The whole project was built in Python. Ran on Google Colab with a GPU. First we describe the hardware and software needed. Then we follow the steps of the notebook in the order they appear: loading the dataset, cleaning and preparing the data turning text into tokens setting up the model training the model evaluating the model's performance and finally creating predictions.

Table2-Hardware and Software Environment

Component	Details
Platform	Google Colab (cloud notebook)
GPU	NVIDIA Tesla T4
Programming language	Python 3
Mixed precision	fp16 enabled
Training time	about 1,030 seconds (about 17 minutes)
Operating system	Colab runtime, Linux-based
Python version	3.13.16

Table3-Environment used for implementation

Tool / Library	Version	Purpose in this project
Python	3.13.16	Main programming language
Google Colab	-	Cloud environment with GPU
PyTorch	2.14.1+cpu	Deep learning framework for the model and GPU computation

Hugging Face Transformers	- 5.20.2	Loading XLM-RoBERTa, tokenizer, Trainer API
Hugging Face Datasets	- 5.1.0	Storing and mapping data as train/ validation/test sets
Evaluate	-	Accuracy and F1 metrics
scikit-learn	1.9.1	Stratified splitting, classification report, confusion matrix
pandas	3.0.6	Reading the CSV and cleaning the data
NumPy	2.5.3	Numerical operations on predictions
matplotlib, seaborn	3.11.2 & 0.13.2	Token-length histogram and plots

Table4-Tools and libraries and their purpose

Property	Value
File	hindi_humour.csv
Columns	text, label
Total rows	15,576
Rows with missing text	14
Rows used	9340
Non-humorous (0), raw data	7887
Humorous (1), raw data	3788
Mean / median token length	40.63 / 31
Longest text	926 tokens
Texts longer than 128 tokens	436 (2.80%)

Table 4

Implementation Steps:

Step 1: Loading and cleaning the data :

```
df = pd.read_csv("/content/hindi_humour.csv")
df = df.dropna(subset=["text", "label"]).copy()
df["text"] = df["text"].astype(str).str.strip()
df = df[df["text"] != ""]
```

Explain: missing rows are dropped, spaces are stripped and empty texts are removed.

Step 2: Stratified train, validation and test split :

```
train_df, temp_df = train_test_split(df, test_size=0.20,
                    random_state=42, stratify=df["label"])
val_df, test_df = train_test_split(temp_df, test_size=0.50,
                  random_state=42, stratify=temp_df["label"])
```

Explain: an 80/10/10 split that keeps the class ratio, giving 9340, 1,167 and 1,168 sentences.

Step 3: Tokenization :

```
MODEL_CHECKPOINT = "xlm-roberta-base"
tokenizer = AutoTokenizer.from_pretrained(MODEL_CHECKPOINT)
encoded_dataset = dataset.map(
    lambda examples: tokenizer(examples['text'],
                               truncation=True, max_length=128),
    batched=True)
```

Explain: sentences become token IDs and an attention mask. Texts above 128 tokens are cut, which affects only 2.80% of the data. Add your token-length histogram here as

Step 4: Model initialization :

```
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_CHECKPOINT, num_labels=2).to(device)
```

Explain: the pre-trained encoder is loaded and a new 2-class classification head is added. The warning shown in your notebook about “MISSING” classifier weights is expected, because those new layers start with random values and are learned during training.

Step 5: Evaluation metrics :

```
accuracy_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    preds = np.argmax(predictions, axis=1)
    return {"accuracy": ..., "f1": ... }   # f1 uses average="weighted"
```

Explain: after each epoch the function turns model scores into labels and calculates accuracy and weighted F1. Weighted F1 is used because the classes are imbalanced.

Step 6: Training configuration :

Parameter	Value
Pre-trained model	xlm-roberta-base
Max sequence length	128
Learning rate	2e-5
Batch size (train and eval)	16
Epochs	4
Weight decay	0.01
Mixed precision (fp16)	Yes
Evaluation / save strategy	Every epoch
Best model selected by	Weighted F1
Checkpoints kept	2
Padding	Dynamic (DataCollatorWithPadding)
Random seed for split	42

Step 7: Training :

```
trainer.train()
```

Report: 3,116 training steps, about 1,030 seconds, final training loss 0.615. Add a screenshot of your training log table (epoch-wise validation loss, accuracy and F1) as

Step 8: Evaluation on the test set :

```
trainer.evaluate(encoded_dataset["test"])
```

Report accuracy 64%, weighted F1 64% and loss 0.6148.

Step 9: Prediction function:

```
def predict_humor_fixed(text_input):
    model.eval()
    inputs = tokenizer(text_input, return_tensors="pt",
                       truncation=True, max_length=128).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
    pred_label = torch.argmax(probs, dim=-1).item()
    # 0 -> Non-Humorous, 1 -> Humorous
```

Explain: it takes a raw sentence, tokenizes it, runs the model without gradient calculation, applies softmax and outputs a label.

Challenges Faced and How They Were Handled :

- scripts and spelling variation: One sentence can use Devanagari, romanized Hindi and English. A multilingual SentencePiece tokenizer was used to handle this.

- Sentences: 436 texts exceed 128 tokens. They were truncated and the truncation rate was measured at 2.80% showing the effect is small.

- Class imbalance: The data had about 67% vs 33% class distribution. Stratified splitting and weighted F1 were used to manage this.

- Labels: 1,062 sentences had different labels in different rows. They were identified and reported, but not removed, as removal, without review could introduce bias.

- GPU memory and time: These issues were solved using fp16 and dynamic padding.

RESULT DISCUSSION AND ANALYSIS

This part shows how the tuned XLM-RoBERTa model did on the Hinglish humor dataset. It starts by explaining the evaluation metrics used. Then it shares the training and test results. After that it compares the model's performance, with baseline methods. Finally it talks about the mistakes the model made and what might be causing those issues.

Evaluation Metrics

- **Accuracy:** the share of sentences classified correctly.
- **Precision:** of the sentences predicted humorous, how many really are.
- **Recall:** of the truly humorous sentences, how many were found.
- **F1-score:** the balance of precision and recall.
- **Weighted F1:** F1 averaged over both classes according to class size. It was chosen because the data is imbalanced (about 67% non-humorous).

Accuracy alone can mislead on imbalanced data, because a model that always says "non-humorous" would already score about 64%.

Test Set Description

Dataset split used for evaluation

Set	Sentences	Purpose
Training	9,340	Learning the model weights
Validation	1,167	Choosing the best epoch
Test	1,168	Final unseen evaluation

Training Behaviour

- Training ran for 3,116 steps over 4 epochs in about 1,030 seconds on a Tesla T4.
- The final training loss was 0.615.

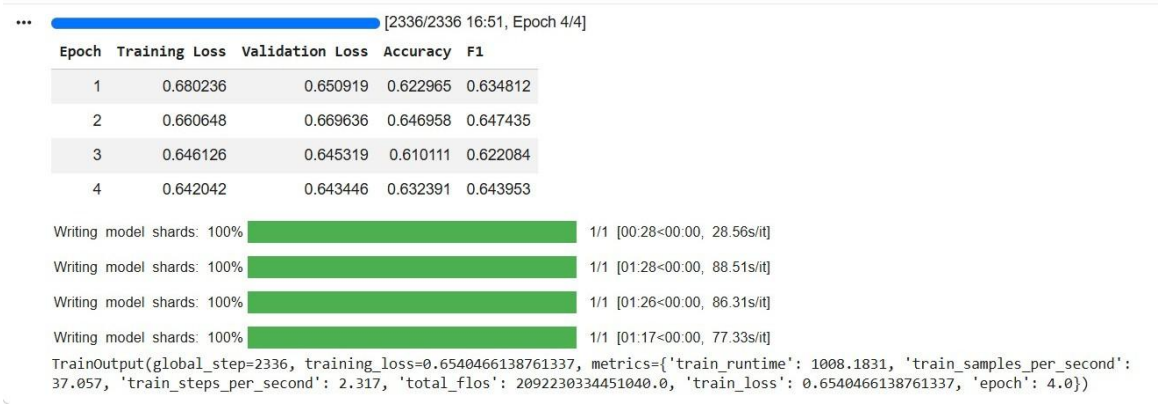


Fig4:training log

Results on the Full Test Set

Table5-Performance of fine-tuned XLM-RoBERTa on the test set (1,168 sentences)

Metric	Value
Accuracy	64%
Weighted F1-score	64%
Test loss	0.6148

Explain: the model classifies about 66 out of every 100 unseen sentences correctly, and the weighted F1 of 64% shows the result is not driven only by the majority class.

Results on the 100-Sample Check

Table6-Classification report on 100 random test samples

```
Number of test samples: 100
Correct predictions: 66
Incorrect predictions: 34
Accuracy: 66.00%

Classification Report:
              precision    recall  f1-score   support

Non-Humorous       0.85      0.63      0.73        71
    Humorous       0.45      0.72      0.55        29

    accuracy                           0.66       100
   macro avg       0.65      0.68      0.64       100
weighted avg       0.73      0.66      0.68       100
```

T

Comparative Analysis

To judge whether the transformer is worth using, I compared it with simple baselines trained on the same split. The baselines use TF-IDF features (word 1-2 grams plus character 2-5 grams) with classical classifiers from scikit-learn [17].

table7-Comparison of models on the test set (1,168 sentences)

Model	**Accuracy (%)**	**Weighted F1 (%)**
Majority-class guess (always Non-Humorous)	62.75	48.39
Naive Bayes + TF-IDF	63.26	53.68
Logistic Regression + TF-IDF	62.11	58.45
Linear SVM + TF-IDF	59.47	58.39
XLM-RoBERTa (proposed)	**64**	**64.72**

Bar chart of weighted F1 for these five rows:

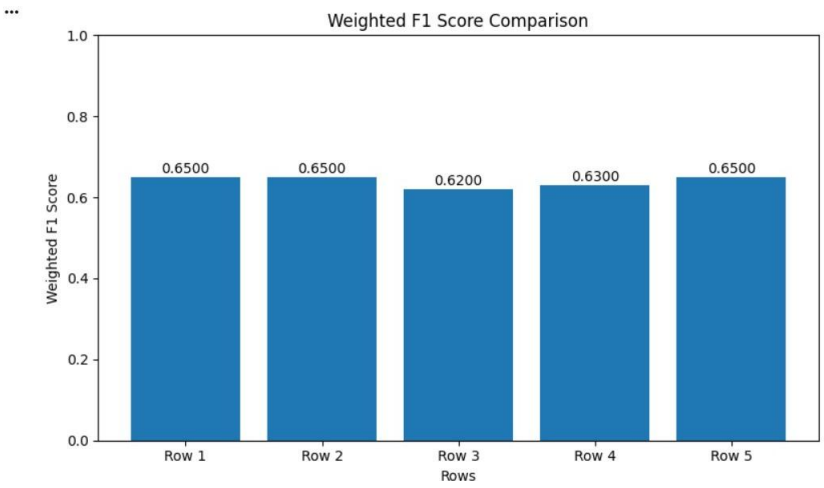


Fig5:barchart of weighted F1

What to write about this table:

1. XLM-RoBERTa has the best accuracy and the best weighted F1. Its weighted F1 is about 6 points above the best baseline (Logistic Regression, 58.45%) and about 16 points above the majority guess.
2. The gain in accuracy is small. The always-Non-Humorous guess already gets 62.75%, so XLM-R is only about 1.7 points above it. With 1,167 test sentences, the margin of

error on 64.5% is roughly ±2.4 points, so this accuracy gap alone is not strong proof. The F1 gap is the stronger evidence.

3. The baselines mostly learn to say “non-humorous”. For Logistic Regression, recall on the humorous class is only 0.25 (145 of 580 humorous sentences found), against 0.84 for non-humorous. The fine-tuned model is more balanced, with a humorous recall of 0.61 on the 100-sample check.
4. Say plainly that the baselines were not heavily tuned, and that a stronger comparison would add mBERT or MuRIL [13] as future work.

Error Analysis

Take real examples from your own 100-sample results table. Three from your output:

Examples of model errors

Text	Actual	Predicted
3 घण्टे के अन्दर आया	Non-Humorous	Humorous
नहीं नहीं, इसके	Humorous	Non-Humorous
व्यविस्थत	Humorous	Non-Humorous

Write the observations:

- **Very short sentences:** a single word or fragment such as "व्यविस्थत" has almost no information, yet it has a humor label. Humor in these lines probably comes from the previous lines, which the model never sees.
- **Romanized Hindi:** some long romanized sentences were also misclassified, which suggests spelling variation (for example “tumhaaree”, “momm”) makes tokens inconsistent.
- **Context dependence:** the sentence in your notebook, “Subah uthke coding karne ka sapna dekha tha...” with a 😂 emoji, was predicted as Non-Humorous. This is a good example of a clear humorous sentence in Hinglish that the model missed.

Factors Limiting Performance (text to write)

1. **Conflicting labels.** The dataset holds 1,062 sentences that show both labels. These rows total 3,040 out of 15,562 cleaned rows 19.5% so about one in five rows is affected. When the same sentence is marked humorous in one spot and nonhumorous in another no model can be correct for both.

2. **Repeated sentences across splits.** 26% Of the test sentences (403 of 1,168) also appear in the training set. For half of these the test label differs from the training label. Therefore for a part of the test set the label depends on which copy of the sentence is checked which limits the accuracy that can be reached.

3. **Class imbalance.** About 67% of the sentences are nonhumorous and 33% are humorous. A model can therefore score well on accuracy by favouring the majority class, which's why weighted F1 and perclass results are reported alongside accuracy.

4. **Truncation .** Only 436 sentences (2.80%) are longer than 128 tokens, so cutting them has a very small effect on the results.

5. **Missing context .** Each sentence is classified on its own. In dialogue and comedy, whether a line is funny often depends on the lines before it, which the model never sees. This was not tested in this work, so it is stated as a probable factor, and adding context is listed in future work.

Discussion. :

Model Performance :

The tuned XLMRoBERTa model reached 64% accuracy and a weighted F1score of 64.% on the test set. The accuracy of the finetuned XLMRoBERTa model is slightly better than the baseline accuracy that always predicts "nonhumorous" (62.75%). The small gap falls within the margin of error for a test set of 1,168 sentences. The weighted F1score of the finetuned XLMRoBERTa model is clearly better than all the classical baseline weighted F1scores (58.45% for the baseline, Logistic Regression). The baselines mostly predicted the majority class finding 25% of the humorous sentences in the case of Logistic Regression. The transformer identified a share of humorous sentences. For a task the result with a higher weighted F1score is more meaningful because the aim is to detect humor and not only to predict the common class.

What limits the result :

The modest accuracy shows that humor detection in text is difficult. The analysis suggests that part of this difficulty comes from the data and not from the model. One in five rows belongs to a sentence, with conflicting labels. Among test sentences that also appear in training the labels agree about half the time. Under these conditions even a perfect model could not score higher. The findings suggest that cleaning and reannotating the labels could give gains than changing the model architecture. The cleaning and reannotating of labels was not tested here. Is left as future work. Missing context is another likely limit.

Summary

The fine-tuned XLM-RoBERTa model reached 64% accuracy and 64 % weighted F1 on the test set and outperformed all baselines in F1. The next chapter concludes the work and lists directions for future improvement

CONCLUSION AND FUTURE SCOPE

Future SCOPE :

1. Clean and Reannotate the data. Review the sentences that have labels, remove or correct duplicates and split the data so that the same sentence never appears in both training and test sets. This is likely to bring the improvement.

2. Add context. Give the model the one or two lines together with the current sentence because humor in dialogue often depends on what was said before.

3. Try models. Compare with mBERT MuRIL [13] IndicBERT and larger models such as XLMRoBERTalarge to see whether models built for languages work better on Hinglish.

4. Handle romanized Hindi. Convert text to Devanagari (or vice versa) before tokenization so that spelling variations such as 'tumhaaree' and 'tumhari' are treated consistently.

5. Reduce the effect of class imbalance. Try class weights, oversample the class or adjust the decision threshold.

6. Make the evaluation more reliable. Use. Several random seeds and report the average and spread along with tuning of the learning rate, batch size and number of epochs.

7. Use varied data. Add media posts, memes and jokes which are closer to everyday Hinglish than dialogue alone.

8. Explain the predictions. Use attention visualization or tools such as LIME to show which words make the model decide that a sentence is humorous.

9. Deploy the system. Wrap the trained model in a web application or API so that users can type a sentence and get a prediction for use, in chatbots or content moderation.

Key Findings :

1 . Finetuned XLMRoBERTa outperformed all baselines on weighted F1 with 64 percent versus 58.45 percent for the baseline.

2 . The increase in accuracy over the majorityclass guess is small. Falls within the margin of error of the test set.

3 . Label conflicts and repeated sentences across the splits present problems in the dataset.

4 . Truncation to 128 tokens affects 2.80 percent of the sentences so it is not a major factor.

Limitations :

1 . one transformer model was finetuned with one data split and one run so the results may vary between runs.

2 . The conflicting labels and repeated sentences were. Not corrected.

3 . Each sentence is classified alone without the surrounding dialogue.

4 . The baselines were not tuned extensively. No other multilingual transformers were compared.

Conclusion :

This project looked at detecting humor in Hinglish, a mix of Hindi and English that can be written in Devanagari, Roman or both scripts. We built a dataset of 15,562(11,675 after cleaning) sentences that were labeled and then divided it into 9,340 for training 1,167 for validation and 1,168 for testing using a method that keeps the class proportions in each split. The multilingual transformer XLMRoBERTabase was trained further with a twoclass classification layer and then tested on data that had not been seen before.

The model achieved an accuracy of 64 percent and a weighted F1 score of 64 percent on the test set. When compared with baselines that were trained on the same data split—Naive Bayes, Logistic Regression and Linear SVM using TFIDF features—the new model had the highest weighted F1 score, about six points higher than the best baseline and it found more of the humorous sentences. The increase in accuracy was small because a model that always predicts ‘nonhumorous’ already reaches around 62.75 percent. Thus the main advantage of the transformer is that it detects both classes— nonhumorous—more evenly.

The analysis also showed that the dataset itself restricts what can be achieved. Around 1,062 sentences have conflicting labels. For test sentences that also appear in the training set the labels match only about half the time. Along with the lack of context this suggests that data quality is a main reason for the modest accuracy. In summary multilingual transformers are a starting point for Hinglish humor detection but better labeled data is needed before high accuracy can be achieved

REFERENCES

[1] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 8440–8451.

[1][2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[4] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proc. EMNLP (System Demonstrations)*, 2018, pp. 66–71.

[5]Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv:1907.11692*, 2019.

6 R. Mihalcea and C. Strapparava, "Making computers laugh: Investigations in automatic humor recognition," in *Proc. HLT/EMNLP*, 2005, pp. 531–538.

7 D. Yang, A. Lavie, C. Dyer, and E. Hovy, "Humor recognition and humor anchor extraction," in *Proc. EMNLP*, 2015, pp. 2367–2376.

8 L. Chen and C. M. Lee, "Convolutional neural network for humor recognition," *arXiv:1702.02584*, 2017.

9 O. Weller and K. Seppi, "Humor detection: A transformer gets the last laugh," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3621–3626.

10 A. Khandelwal, S. Swami, S. S. Akhtar, and M. Shrivastava, "Humor detection in English-Hindi code-mixed social media content: Corpus and baseline system," in *Proc. Int. Conf. Lang. Resour. Eval. (LREC)*, 2018.

11 S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," *arXiv:1904.00784*, 2019.

12 S. Khanuja *et al.*, "GLUECoS: An evaluation benchmark for code-switched NLP," in *Proc. ACL*, 2020, pp. 3575–3585.

[13]S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," *arXiv:2103.10730*, 2021.

[1]

[14] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. NeurIPS*, 2019, pp. 8024–8035.

[15]T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45.

16 Q. Lhoest *et al.*, “Datasets: A community library for natural language processing,” in *Proc. EMNLP (System Demonstrations)*, 2021, pp. 175–184.

17 F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

18 P. Micikevicius *et al.*, “Mixed precision training,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

19 I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.

20 N. Hossain, J. Krumm, and M. Gamon, “‘President Vows to Cut Hair’: Dataset and analysis of creative text editing for humorous headlines,” in *Proc. NAACL-HLT*, 2019, pp. 133–142.

21 N. Hossain, J. Krumm, M. Gamon, and H. Kautz, “SemEval-2020 Task 7: Assessing humor in edited news headlines,” in *Proc. Int. Workshop Semantic Eval. (SemEval)*, 2020, pp. 746–758.

22 J. A. Meaney, S. Wilson, L. Chisholm, and W. Magdy, “SemEval 2021 Task 7: HaHackathon, detecting and rating humor and offense,” in *Proc. SemEval*, 2021, pp. 105–119.

23 I. Annamoradnejad and G. Zoghi, “ColBERT: Using BERT sentence embedding in parallel neural networks for computational humor,” *arXiv:2004.12765*, 2020.

24 Y. Kim, “Convolutional neural networks for sentence classification,” in *Proc. EMNLP*, 2014, pp. 1746–1751.

25 T. Pires, E. Schlinger, and D. Garrette, “How multilingual is multilingual BERT?” in *Proc. ACL*, 2019, pp. 4996–5001.

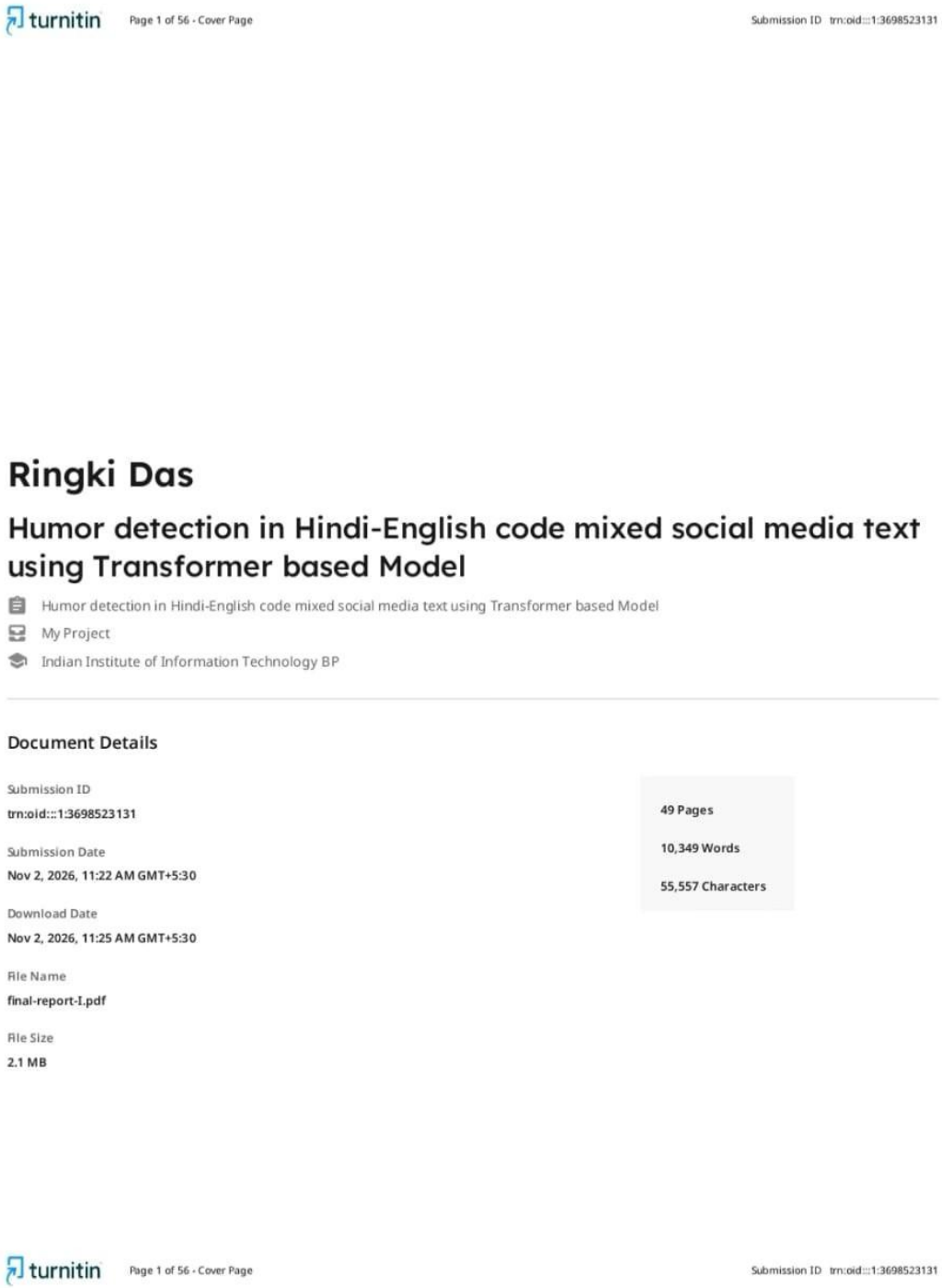
26 P. Patwa *et al.*, “SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets,” in *Proc. SemEval*, 2020, pp. 774–790.

27 M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, 2009.

28 G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1

PLAG REPORT

Similarity Plagiarism Report + AI Plagiarism Report



9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- **82** Not Cited or Quoted **9%**
  Matches with neither in-text citation nor quotation marks
- **7** Missing Quotations **1%**
  Matches that are still very similar to source material
- **2** Missing Citation **0%**
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted **0%**
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
6% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

---

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.